COMPONENT BASED SOFTWARE ENGINEERING

Specification of Components

Meaning of component specification;

- A **component specification** is essentially the **blueprint** of a software component. It defines:
- **Purpose** → What the component is supposed to achieve.
- **Behavior** → How it should act under different conditions.
- **Interfaces** → How it interacts with other components (inputs, outputs, protocols).
- **Constraints** → Any limitations such as performance requirements, resource usage, or compatibility rules.
- Think of it like the **instruction manual** for a Lego block in a larger system: it tells you exactly what the block does, how it connects to others, and what rules must be followed so everything fits together smoothly.

Component specification-Intro

- Component specification refers to the detailed description of the functionalities, features, and requirements of a software component. A software component is an independent module that performs a specific task or set of tasks within a larger software system.
- Component specification defines the interface, behavior, and constraints of a component and provides a blueprint for its development, testing, and integration into the system.
- Component specification is an essential part of the software development process, as it helps to ensure that the components are designed and implemented correctly and meet the system's requirements.
- It provides a clear understanding of the component's purpose, functionality, and dependencies, which helps developers to build software that is modular, scalable, and maintainable.

cont:.:

- The component specification typically includes the following information:
- **Functional requirements:** The specific tasks that the component must perform and the expected outcomes.
- **Non-functional requirements:** The performance, reliability, security, and usability requirements of the component.
- **Interface requirements:** The input and output parameters, data structures, and communication protocols used by the component.
- **Dependencies:** The other components or systems that the component relies on.
- **Constraints:** The limitations or restrictions on the component's design and implementation, such as hardware or software compatibility issues.

Cont.:

- Component specification is usually created by software architects or system analysts, who work closely with stakeholders to identify the system's requirements and design the component's architecture.
- The specification is then reviewed and approved by the development team , who use it as a reference for building and testing the component.
- Any changes or updates to the specification are documented and communicated to all stakeholders to ensure that everyone is on the same page.

Cont.:

- Overall, component specification is a critical aspect of software development that helps to ensure that the components are designed and implemented correctly and meet the system's requirements.
- It provides a clear and detailed description of the component's purpose, functionality, and constraints, which helps developers to build software that is modular, scalable, and maintainable.

Cont.:

- Component-based software development (CBSD) builds applications using independent, reusable software building blocks instead of writing everything from scratch.
- Component requirements describe what these pieces must do and how they must behave to fit together.
- Component requirements fall into a few key areas that guide how you pick, build, and connect software pieces.

Component Requirements

- **Component requirements** are the functional and non-functional specifications that a software component must satisfy so that it can be developed, reused, integrated, and maintained within a software system.

Functional Requirements

- **Specific Services:** Define the exact tasks or business logic a component handles, such as processing a payment or logging a user in.
- **Provided Interfaces:** Specify the methods, inputs, and outputs a component offers for other parts of the system to use.
- **Required Interfaces:** List any external services or dependencies the component needs from other components to do its job

Non-Functional Requirements

- **Independence:** Ensure the component operates on its own with minimal reliance on specific outside code.
- **Scalability:** Require individual components to handle varying workloads or scale up independently.
- **Reliability and Quality:** Set standards for how stable the component must be, since system quality depends on reliable parts

Integration and Design Requirements

- **Standardization:** Require the component to follow a standard model for metadata, communication, and deployment.
- **Composable Contracts:** Demand that all interactions happen strictly through public interfaces so internal code can stay hidden.
- **Adaptability:** Ensure components can be adjusted or fitted with glue code to prevent architectural mismatches

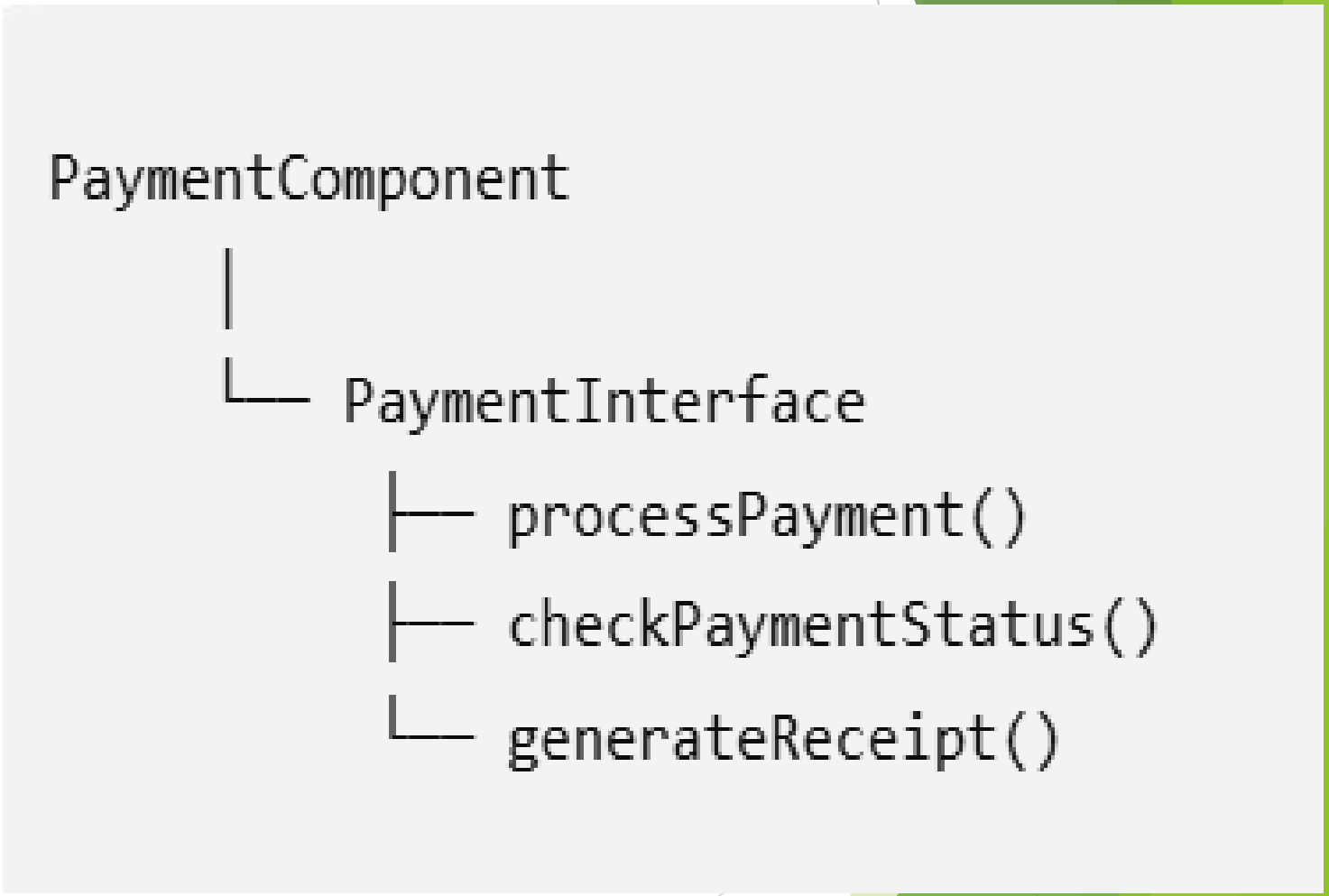
Provided Interfaces

- a **provided interface** is the set of services, operations, or functionalities that a software component **offers to other components or applications**.
- It defines **what the component can provide**, without necessarily revealing how those services are implemented.

Example

Consider a Payment Component that provides

- Other components can call these operations through the **PaymentInterface** without knowing the internal implementation of the payment component.



Characteristics of a Provided Interface

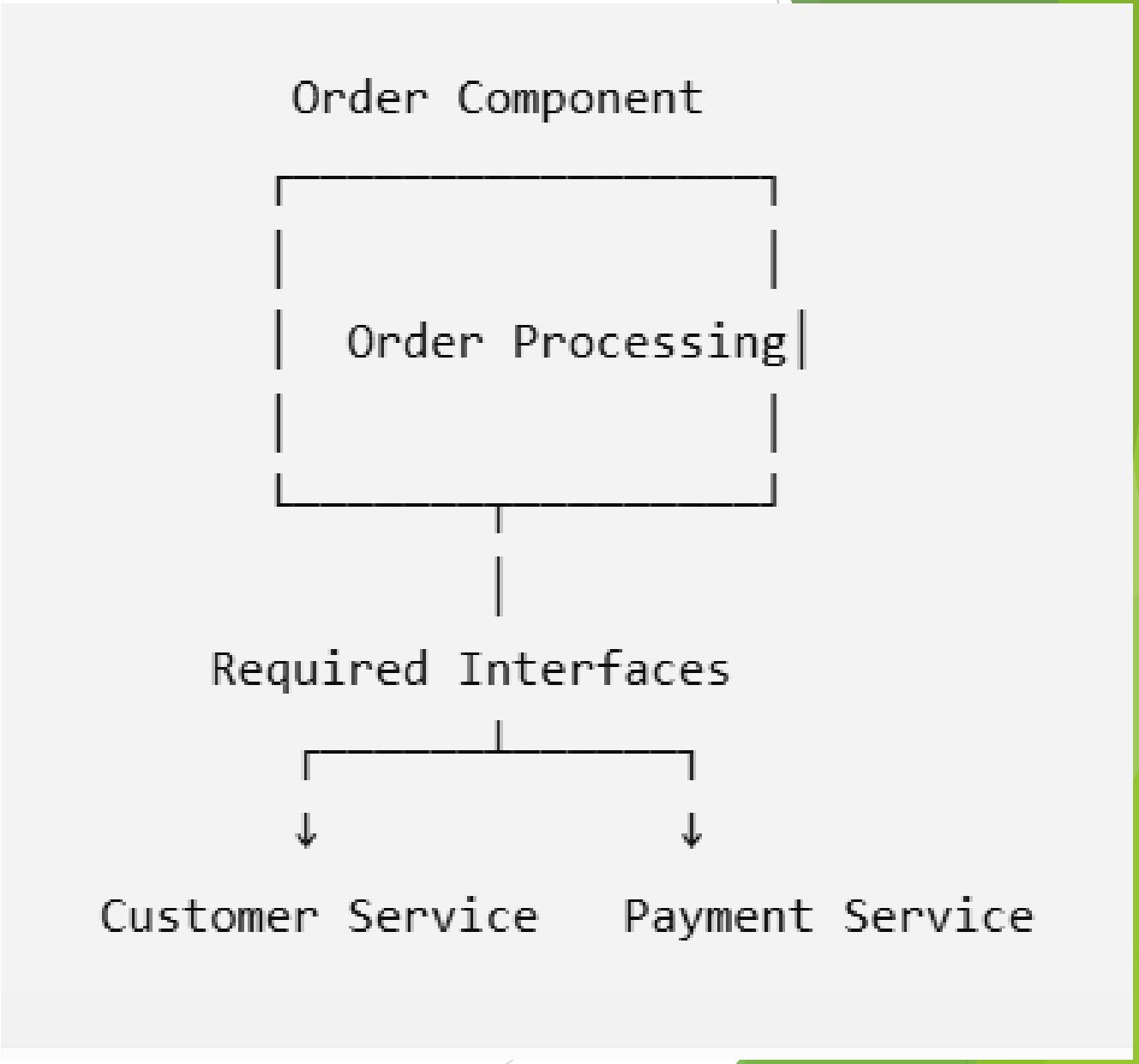
- **Defines available services** – specifies what functionality the component makes available.
- **Specifies operations/methods** – identifies the functions that other components can invoke.
- **Hides implementation details** – users of the interface do not need to know how the service is implemented.
- **Supports component integration** – allows other components to interact with the component in a standard way.
- **Promotes reusability** – components can be reused as long as they provide the required interface.
- **Defines input and output** – specifies parameters accepted and values or responses returned.

Required Interfaces

- A **required interface** is a set of services or operations that a software component **needs from other components** in order to perform its functions

Example

- Suppose we have an **Order Component** that needs customer information and payment services:



Cont.:

- The Order Component may require:
  - getCustomerDetails()
  - validateCustomer()
  - processPayment()
  - checkPaymentStatus()
- The Order Component **does not provide these services itself**. It depends on other components that provide them.

Characteristics of Required Interfaces

- **Identify dependencies** – show services that a component needs from other components.
- **Define expected services** – specify the operations the component expects to access.
- **Support component integration** – connect a component to services provided by other components.
- **Promote loose coupling** – the component depends on an interface rather than the internal implementation of another component.
- **Enable component substitution** – another component can provide the required services as long as it implements the expected interface.

Key Roles of Interfaces in CBSE

- **Enabling Parallel Development:** Clearly defined interfaces allow distinct software engineering teams to develop separate components concurrently. As long as the interface contract remains stable, internal changes will not disrupt parallel workflows.
- **Interchangeability and Upgrades:** Since a system depends on the interface and not the internal mechanics, any component can be cleanly replaced or updated with a newer version—provided it complies with the established interface rules.
- **Facilitating System Composition:** During integration, software engineers construct systems by connecting the "required" interface of one component directly to the "provided" interface of another.
- **Mitigating Incompatibility:** In cases where two third-party components must interact but feature mismatching signatures, developers introduce **adapters**. The adapter acts as a bridge, translating calls to resolve the interface conflict

End

!!!!!!!